

AutoPT: How Far Are We from the End2End Automated Web Penetration Testing?

BENLONG WU, University of Science and Technology of China, China
GUOQIANG CHEN, QI-ANXIN Technology Research Institute, China
KEJIANG CHEN*, University of Science and Technology of China, China
XIUWEI SHANG, University of Science and Technology of China, China
JIAPENG HAN, Chaitin Future Technology Co., Ltd, China
YANRU HE, University of Science and Technology of China, China
WEIMING ZHANG, University of Science and Technology of China, China
NENGHAI YU, University of Science and Technology of China, China

Penetration testing is essential to ensure Web security, which can detect and fix vulnerabilities in advance, and prevent data leakage and serious consequences. The powerful inference capabilities of large language models (LLMs) have made significant progress in various fields, and the development potential of LLM-based agents can revolutionize the cybersecurity penetration testing industry. In this work, we establish a comprehensive end-to-end penetration testing benchmark using a real-world penetration testing environment to explore the capabilities of LLM-based agents in this domain. Our results reveal that the agents are familiar with the framework of penetration testing tasks, but they still face limitations in generating accurate commands and executing complete processes. Accordingly, we summarize the current challenges, including the difficulty of maintaining the entire message history and the tendency for the agent to become stuck.

Based on the above insights, we propose a **Penetration testing State Machine (PSM)** that utilizes the Finite State Machine (FSM) methodology to address these limitations. Then, we introduce **AutoPT**, an automated penetration testing agent based on the principle of PSM driven by LLMs, which utilizes the inherent inference ability of LLM and the constraint framework of state machines. Our evaluation results show that AutoPT outperforms the baseline framework ReAct on the GPT-4o mini model and improves the task completion rate from 22% to 41% on the benchmark target. Compared with the baseline framework and manual work, AutoPT also reduces time and economic costs further. Hence, our AutoPT has facilitated the development of automated penetration testing and significantly impacted both academia and industry.

CCS Concepts: • **Security and privacy** → **Penetration testing**.

Additional Key Words and Phrases: Web Penetration Testing, Automation, Large Language Model, AI Agent

*Corresponding author

Authors' addresses: Benlong Wu, University of Science and Technology of China, HeFei, China, dizzylong@mail.ustc.edu.cn; Guoqiang Chen, QI-ANXIN Technology Research Institute, Beijing, China, guoqiangchen@qianxin.com; Kejiang Chen, University of Science and Technology of China, HeFei, China, chenkj@ustc.edu.cn; Xiuwei Shang, University of Science and Technology of China, HeFei, China, shangxw@mail.ustc.edu.cn; Jiapeng Han, Chaitin Future Technology Co., Ltd, HangZhou, China, jiapeng.han@chaitin.com; Yanru He, University of Science and Technology of China, HeFei, China, heyuanru@mail.ustc.edu.cn; Weiming Zhang, University of Science and Technology of China, HeFei, China, zhangwm@ustc.edu.cn; Nenghai Yu, University of Science and Technology of China, HeFei, China, ynh@ustc.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

XXXX-XXXX/2024/11-ART \$15.00

<https://doi.org/XXXXXXX.XXXXXXX>

ACM Reference Format:

Benlong Wu, Guoqiang Chen, Kejiang Chen, Xiuwei Shang, Jiapeng Han, Yanru He, Weiming Zhang, and Nenghai Yu. 2024. AutoPT: How Far Are We from the End2End Automated Web Penetration Testing?. 1, 1 (November 2024), 22 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 INTRODUCTION

Web security [53] is a daunting challenge. Penetration testing [52, 60] and red team testing [55] have become necessary means to ensure Web security. For example, in 2024, Bank of America’s data breach occurred, and the bank’s service provider Infosys Mccamish Systems suffered a ransomware attack, resulting in the exposure of sensitive information from more than 60,000 customers¹. However, suppose the company conducts comprehensive penetration testing before launching a new system. In that case, these security vulnerabilities may be discovered and fixed in advance, thus avoiding data leakage and the serious consequences it may cause. Therefore, our study focuses on the field of penetration testing and focuses on automated testing, especially black-box testing [28].

Penetration testing is a way to evaluate Web security by simulating real attacks [6]. It involves a team of security experts assuming the role of attackers, employing tools and techniques like real hackers. This process entails deliberate attack attempts on target systems, networks, or applications to identify and exploit vulnerabilities within these environments. Currently, most penetration tests are labor-intensive processes conducted by skilled professionals who leverage their organizational knowledge and expertise and use semi-automated tools to execute a predefined set of automated operations [14]. A few studies have attempted automated penetration tests, such as rule-based methods [5, 23, 70] and deep reinforcement learning-based solutions [47]. However, none of these automated methods can solve **the end-to-end penetration testing task**, defined as the entire process of completing automated penetration testing without human involvement and that automatically adapts to various environments.

Benchmark. To address this question, we began to explore the capabilities of LLM-based agents in end-to-end automated penetration testing tasks. Unfortunately, current penetration testing benchmarks are not granular enough to perform a fair and granular assessment of the progress made. Among them, CTF-related benchmarks [9, 51] are far from actual penetration scenarios, and HackTheBox [1] mostly belongs to the actual combat of compound vulnerabilities, which is too complex for the current single-agent capabilities. To address this limitation, we built a refined benchmark covering the OWASP’s top 10 vulnerability list [54] via test machines from Vulhub [58]. Then, we performed detailed manual annotations, including task complexity annotations based on the number of exploit steps. In addition, for end-to-end task goal checking, we created an explicit task goal string for each task triggered if the vulnerability exploit goal is met. In this way, our benchmark can meet the needs of end-to-end penetration testing task evaluation.

Motivation. Large language models (LLMs) have developed rapidly and have shown great capabilities in many applications and tasks [15, 21, 39, 61]. Furthermore, LLMs have been applied to tasks that require interaction with the environment through agents [36, 40, 62], such as code execution feedback and real-world scene interaction. Despite the significant efforts of tens of thousands of penetration testing researchers worldwide, fully automated penetration testing has remained challenging for an extended period [2, 62]. Recently, several studies have aimed at helping humans perform penetration testing, such as PentestGPT [13]. Nevertheless, they necessitate extensive human-computer interaction and lack a systematic and quantitative evaluation of current LLM-based agents on end-to-end Web penetration testing tasks. Therefore, the following question arises: *How far are we from the end-to-end automated Web penetration testing?*

¹<https://www.anquanke.com/post/id/293251>

Based on the benchmark we built, we conducted an end-to-end evaluation of the existing to pave the way for subsequent research. First, we tried many advanced models, from which we selected GPT-3.5, GPT-4o, and GPT-4o mini models that passed the first pre-experiment as representative LLMs for subsequent research. Then, we designed an end-to-end testing strategy, which includes carefully designed prompts to guide the agent to conduct penetration testing. For existing agent frameworks, we selected the ReAct [65] framework and the framework built on the PentestGPT [13] core penetration testing task tree (PTT) as a representative framework. Each agent receives prompts and black box information from the target machine, which spontaneously queries the environmental information, infers subsequent operations, executes terminal commands, and operates browsers via controlled tools. This process is repeated until the LLM autonomously completes the penetration testing. Finally, we compare its results with the baseline solution of officially certified penetration testers [58]. By analyzing the reasons for the agent's failure cases, we summarize the main challenges of current intelligent agents performing end-to-end penetration testing tasks as follows: 1) Maintaining the entire message history is difficult due to model context size limitations. 2) The agent may get stuck on subtle problems during self-iteration, leading to task failure. 3) Current model inference capabilities restrict an agent from completing this task.

Our Methodology. To address these challenges, we introduce a classic method, the **Finite State Machine (FSM)**, which enables us to better manage the agent's decision-making process by maintaining a clear and structured sequence of actions while retaining the model's own operation space to the maximum extent. We developed a novel agent architecture called the **Penetration Testing State Machine (PSM)**, which draws inspiration from the traditional FSM.

In traditional Web security tasks [25], penetration testers often have some fixed actions, such as data query and reflection inspection. Based on this, we divide the PSM into the Agent state and the Rule state, constraining the workflow of solving tasks and guiding subsequent operations to solve the end-to-end penetration testing task. We launched **AutoPT (Automated Penetration Testing)**, an end-to-end system based on PSM designed to increase the use of agents in this field. AutoPT draws inspiration from the collaborative dynamics common in real-world human penetration testing teams [26]. It uses the third-party architecture LangChain [10] to build an agent, including vulnerability scanning, selection, reconnaissance, exploitation, and check states. Each state reflects each part of the penetration testing process. What is unique is that this architecture can clearly and visually display the state jumps of the entire state machine, thereby improving the efficiency and success rate of penetration testing. Specifically, our method contains the following states:

- The *Scanning* state uses an open-source scanner to obtain a list of system vulnerabilities.
- The *Selection* state follows the thinking of general infiltrators, formats the list of vulnerabilities according to the results of the *Scanning* state and selects the most likely vulnerability from it.
- The *Reconnaissance* state uses tools to scout based on vulnerability information
- The *Exploitation* state simulates a junior penetration tester and faithfully attempts to exploit vulnerabilities based on the results of the vulnerability query.
- The *Check* state makes detection jumps based on the output value of the vulnerability attempt.

Overall, these states work as an integrated system. AutoPT completes the initial end-to-end penetration testing task by combining advanced strategies and precise execution, thereby maintaining a coherent and effective testing process. We created a github repository that includes all benchmark entries and environments, the code used to implement the pre-experiments, and the AutoPT system. The entire project will be open-sourced after peer review. For more information, please refer to Section 9.

We evaluate AutoPT in different test scenarios to validate its effectiveness and efficiency. In our proposed benchmark, AutoPT significantly outperformed direct applications of both the ReAct

and improved PTT frameworks, increasing task completion rates from 22% to 41%, respectively. The execution efficiency was improved by 96.7%, and the total cost of using the OpenAI API was decreased by 71.6%. We believe this is because AutoPT reduces the context width requirement of each state model by decomposing the end-to-end task into multiple subtasks, thus compensating for the impact of subtasks and avoiding the failure of the entire task due to the dilemma of a subtask. This evaluation highlights the practical value of AutoPT in improving the efficiency and accuracy of penetration testing tasks.

During the evaluation process, we gained interesting insights into the capabilities and limitations of LLM-based agents in penetration testing. First, in contrast to human behavior, agents can quickly read and query relevant information and make rapid attempts based on vulnerability information. In addition, intelligent agents perform operations such as scanning, reconnaissance, and exploitation according to target requirements. However, we also noticed that current agents are affected by model capabilities and model hallucinations [35, 36] and often output incorrect commands that cause task failures, which is an important aspect of optimizing end-to-end penetration testing goals. We can also see that in the near future, fully automatic penetration testing agents will surely appear in the public eye.

Contribution. Overall, the major contributions of our work are as follows:

- **Develop a fine-grained end-to-end penetration testing benchmark.** We developed a robust and representative penetration testing benchmark with test machines from the leading platform, VulnHub. The benchmark includes 20 out-of-the-box docker environments, covering the OWASP's top 10 vulnerability list, both easy and difficult, and detailed and specific vulnerability targets and detection content for each vulnerability, providing a fair and comprehensive evaluation for penetration testing. To the best of our knowledge, this benchmark is the first to provide a clear evaluation and inspection of end-to-end penetration testing tasks.
- **Design a novel agent framework PSM and implement a novel end-to-end penetration testing system.** We drew inspiration from traditional finite state machines, integrated the design of general penetration tester behavior logic, and built a penetration test state machine. Based on its principles, we implemented a novel end-to-end penetration testing system AutoPT. This architecture optimizes the use of agents and significantly improves the efficiency and effectiveness of automated penetration testing.
- **Comprehensive evaluation and analysis of LLM-driven agents in end-to-end penetration testing tasks.** By adopting the GPT-3.5, GPT-4o, and GPT-4o mini models along with the ReAct and RTT frameworks, our exploratory study rigorously investigates the strengths and limitations of agents in penetration testing. To the best of our knowledge, this is the first systematic and quantitative study of the ability of LLM-based agents to perform end-to-end automated penetration testing. Our results show that LLMs show great potential in advancing automation to complete end-to-end penetration testing tasks. We call for more research in this area to further enhance the capabilities of LLMs so that they can play a more critical role in the complex tasks of penetration testing.

2 BACKGROUND

2.1 Related Work

2.1.1 Penetration Testing. Penetration testing is a critical practice for enhancing the security of the system of an organization. In classic penetration testing, security professionals (known as penetration testers) typically utilize automated or semiautomated tools to analyze target systems. The standard process is divided into six phases [12]: 1) Planning and Reconnaissance; 2) Scanning and

Enumeration; 3) Exploitation; 4) Post-Exploitation Activities; 5) Reporting and Recommendations; 6) Re-testing. These steps help penetration testers systematically evaluate the Web system security.

For a long time, even though tens of thousands of penetration testing researchers around the world have made great efforts, fully automated penetration testing has still been difficult to achieve [5, 31]. Process automation challenges arise from the need to understand the comprehensive knowledge required to filter and exploit various vulnerabilities and the interaction of information between different stages. In addition, penetration testing often requires many tools with different features and specialized functions, which are designed with only human convenience in mind. The diversity of tool usage increases the complexity of the automation process. Therefore, even with the involvement of deep learning [33] and artificial intelligence [29], solving the end-to-end automated penetration testing task is still a difficult problem.

On the other hand, we found that less work has been done on automating end-to-end web penetration testing tasks. The relevant work has focused mainly on system security penetration testing [19, 24, 27]. To date, most penetration tests have been manually orchestrated by human experts, who combine their specialized organizational knowledge and expertise and use semi-automated tools [16, 71] that run a programmatic collection of automated actions. Researchers have explored smarter automation through rule-based methods [22, 70] and deep reinforcement learning [47]. However, none of these automation methods can cover a full set of attack tasks and automatically adapt to various environments.

2.1.2 Large Language Models. The past year has seen tremendous success for large language models, which are powered primarily by Transformer models [68]. Commercial products such as GPT-3.5 [43] and GPT-4 [18], and open source products such as Llama 3 [17] have amassed a large user base. As LLM capabilities have increased, AI agents have become increasingly powerful [34]. In this work, we focus on AI agents that solve complex end-to-end tasks. These agents are now almost exclusively powered by LLMs that support tools [34]. The basic architecture of these agents involves an LLM that is given a task and uses tools through an API to perform that task.

Recent work has explored the application of LLM in the context of system security penetration testing [67, 69]. Happe *et al.* [24] established a command-response loop between LLM and a vulnerable virtual machine, testing privilege escalation only on Linux. They subsequently developed Wintermute [27], improved the design, added three prompt templates, interacted with LLM, and only tested privilege escalation. In addition, PentestGPT [13], a semiautomated framework for web applications, includes parsing, reasoning, and generation modules but requires penetration testers to operate as agents. In this work, we use the ability OpenAI GPT model to study the capabilities of LLM to automate Web Penetration testing tasks.

2.2 Task Definition

End-to-end black box penetration testing is a security assessment method that aims to simulate the perspective of a real attacker to identify security vulnerabilities in a system or website [7]. This process typically starts from an authorized outside perspective, where the tester knows nothing about the internal structure, code, or configuration of the target system or network [49]. In our work, the LLM-based agent simulates a tester similar to a potential attacker, discovering and exploiting possible vulnerabilities through the exposed interfaces and services.

As a preliminary attempt at end-to-end penetration testing, we decided to simplify the experimental objectives. In accordance with the standard process in Section 2.1.1, we considered exploring only the most core issues, thereby simplifying the existing end-to-end penetration testing tasks as follows: 1) Scanning, 2) Reconnaissance, and 3) Exploitation. Other post-penetration, reporting, and retesting processes are not the focus of this study.